Infant Cry State Recognition Using Acoustic Features and Support Vector Machine

Meesala Sree Sai Nath
Dept. of Computer Science & AI
Rishihood University
Sonipat, India
meesala.s23csai@nst.rishihood.edu.in
Roll No: 230103

Akula Jithendranath
Dept. of Computer Science & AI
Rishihood University
Sonipat, India
akula.j23csai@nst.rishihood.edu.in
Roll No: 230120

***Abstract*—Automated recognition of infant cry states has applications in infant care and caregiver assistance. This paper presents an interpretable acoustic classification pipeline for recognizing eight infant states from cry audio. The system follows a four-phase architecture: (1) audio preprocessing including resampling to 16 kHz, peak normalization, and silence trimming; (2) noise augmentation with Gaussian noise, pink noise, and time shifting; (3) extraction of a 106-dimensional feature vector using MFCCs with deltas, spectral centroid, zero crossing rate, and chroma features; and (4) classification using an RBF-kernel Support Vector Machine with balanced class weighting. The system is evaluated on a dataset of 2,108 samples (cleaned and noise-augmented) across eight classes with a 36:1 class imbalance ratio, using Macro F1-Score as the primary metric. The baseline achieves a Macro F1-Score of 0.26 and overall accuracy of 23.70%, establishing a transparent foundation for future improvement.**

***Index Terms*—Infant cry recognition, MFCC, Support Vector Machine, signal processing, noise augmentation, feature extraction.**

I. Introduction

Infant crying is a primary form of pre-linguistic communication. Different cry types—such as hunger, pain, fatigue, and discomfort—carry distinct acoustic signatures that can potentially be classified using machine learning [1]. Automating this classification could assist caregivers in understanding infant needs more quickly.

Recent deep learning approaches, such as ECAPA-TDNN [2] and Blueprint Separable CNN with TF-RNN [3], have shown high accuracy on audio classification tasks. However, these models are computationally expensive and act as black boxes—it is difficult to trace their predictions back to specific acoustic properties of the signal.

In this project, we take a different approach by building a classical machine learning pipeline that is fully transparent. We use handcrafted audio features (MFCCs, spectral centroid, ZCR, chroma) that have known physical meaning, and classify them with a Support Vector Machine (SVM). The key challenge is the severe class imbalance in the dataset (36:1 ratio), which we address through balanced class weighting and Macro F1-Score evaluation.

Our pipeline consists of four phases:

TABLE I
Dataset Class Distribution (After Augmentation)

Cry State	Samples	Proportion (%)
Belly Pain	254	12.05
Burping	236	11.20
Cold/Hot	230	10.91
Discomfort	276	13.09
Hungry	764	36.24
Lonely	22	1.04
Scared	54	2.56
Tired	272	12.90
Total	**2,108**	**100.00**

1) Audio preprocessing (resampling, normalization, silence trimming)
2) Noise augmentation (Gaussian, pink noise, time shifting)
3) Feature extraction (106-dimensional feature vector)
4) SVM classification with RBF kernel and grid search

II. Dataset Description

The dataset consists of audio recordings of infant cries, organized into eight categories representing different physiological and emotional states. The raw dataset contains approximately 1,054 audio files. After preprocessing (Phase I) and noise augmentation (Phase II), the total training corpus expands to 2,108 samples (1,054 cleaned + 1,054 noise-augmented). Table I shows the class distribution of the final feature dataset.

The dataset has a severe class imbalance—*hungry* has 764 samples while *lonely* has only 22, maintaining a 36:1 imbalance ratio. Audio files are in multiple formats (.wav, .mp3, .ogg, .flac, .m4a) with varying sample rates and recording conditions.

III. Methodology

The system follows a four-phase sequential pipeline. Fig. 2 shows the overall flow.

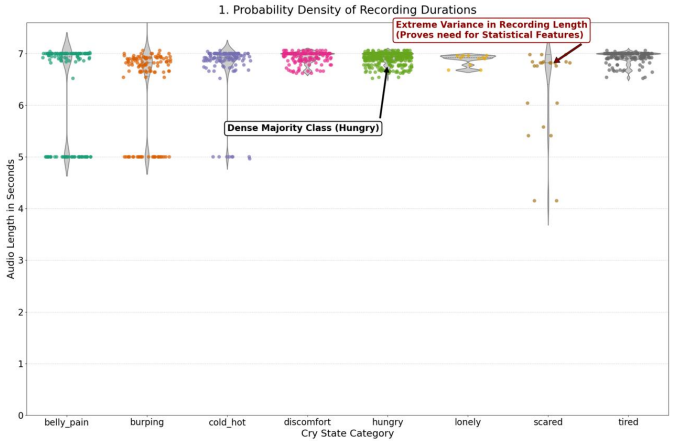

Fig. 1. Class distribution across the 8 infant cry categories.

A. Phase I: Audio Preprocessing

The preprocessing module (`preprocessing.py`) takes raw audio files and produces cleaned, standardized WAV files. Three operations are applied sequentially:

1) Resampling to 16 kHz: All audio is loaded using `librosa.load()` and resampled to a uniform 16 kHz sample rate. This ensures all files have the same temporal resolution regardless of their original recording settings.

2) Peak Normalization: The audio amplitude is scaled to $[-1, 1]$ by dividing by the maximum absolute value:

$$y_{\text{norm}}(t) = \frac{y(t)}{\max(|y(t)|)} \tag{1}$$

This removes differences in volume caused by different microphones or recording distances.

3) Silence Trimming: Leading and trailing silence is removed using `librosa.effects.trim()` with a threshold of 25 dB below peak energy. This ensures feature extraction focuses on the actual cry content.

The cleaned files are saved as 16-bit WAV files in the `data/cleaned/` directory, preserving the class-folder structure.

B. Phase II: Noise Augmentation

The noise augmentation module (`noise_augmentation.py`) creates noisy versions of the cleaned audio to make the model more robust. Three augmentations are applied in sequence to each file:

1) Gaussian White Noise (SNR = 20 dB): Random noise from a normal distribution is added, with noise level controlled by the signal-to-noise ratio:

$$P_{\text{noise}} = \frac{P_{\text{signal}}}{10^{(\text{SNR}_{\text{dB}}/10)}} \tag{2}$$

2) Pink Noise (SNR = 15 dB): Pink noise ($1/f$ spectrum) is generated using FFT-based spectral shaping and added at 15 dB SNR. Pink noise has more energy in lower frequencies, simulating background sounds like fans or traffic.

TABLE II
NOISE AUGMENTATION SETTINGS

Augmentation Type	Parameter	Value
Gaussian White Noise	SNR	20 dB
Pink ($1/f$) Noise	SNR	15 dB
Time Shifting	Max shift	10%

TABLE III
FEATURE VECTOR COMPOSITION (106 DIMENSIONS)

Feature	Library Function	Dims
MFCC	`librosa.feature.mfcc`	26
Δ-MFCC	`librosa.feature.delta`	26
$\Delta\Delta$-MFCC	`librosa.feature.delta`	26
Spectral Centroid	`spectral_centroid`	2
ZCR	`zero_crossing_rate`	2
Chroma	`chroma_stft`	24
Total		**106**

3) Time Shifting (max 10%): The audio is randomly shifted in time by up to 10% of its length using `np.roll()`, with the wrapped portion zero-filled.

Table II summarizes the augmentation settings. The augmented files are saved in `data/noisy/`, one noisy version per original file.

C. Phase III: Feature Extraction

The feature extraction module (`feature_extraction.py`) converts each audio file into a fixed-length vector of 106 features. Features are computed using 25 ms frames (400 samples) with 10 ms hop (160 samples, 60% overlap). For each feature type, the mean and standard deviation across all frames are computed. Table III shows the complete breakdown.

MFCCs (13 coefficients): We extract 13 MFCCs per frame using `librosa.feature.mfcc()`. MFCCs capture the spectral shape of the audio. Delta (Δ) and delta-delta ($\Delta\Delta$) coefficients capture how the spectral shape changes over time. This gives $13 \times 2 \times 3 = 78$ MFCC-related features.

Spectral Centroid: Indicates the "center of mass" of the frequency spectrum. Computed using `librosa.feature.spectral_centroid()` (2 features: mean and std).

Zero Crossing Rate (ZCR): Counts how often the signal crosses zero per frame. High ZCR = noisy sounds; low ZCR = tonal sounds. Computed using `librosa.feature.zero_crossing_rate()` (2 features).

Chroma Features (12 pitch classes): Projects spectrum onto 12 pitch classes (C through B), capturing tonal content. Computed using `librosa.feature.chroma_stft()` (24 features: mean and std for each pitch class).

Features are extracted from both `data/cleaned/` and `data/noisy/` directories, producing a total of 2,108 feature vectors saved as `data/features/features.csv`.

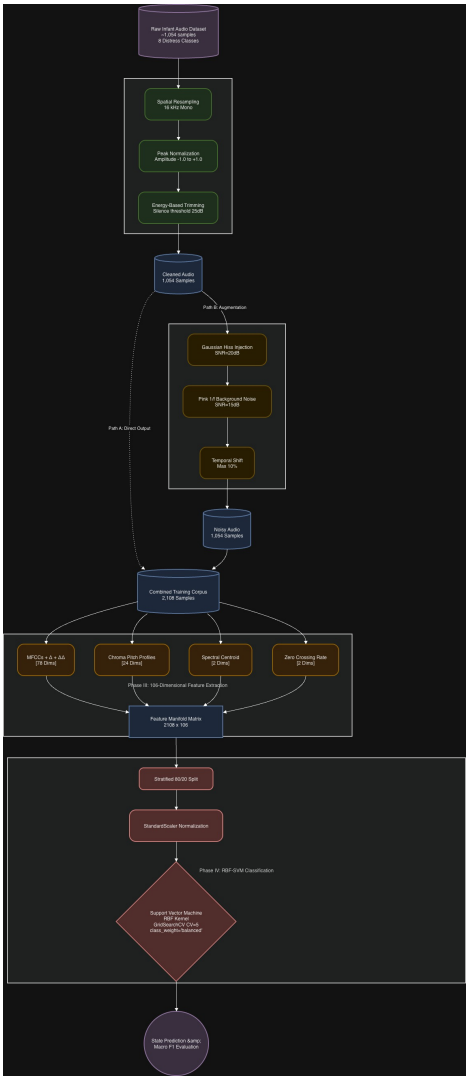

Fig. 2. End-to-end pipeline architecture.

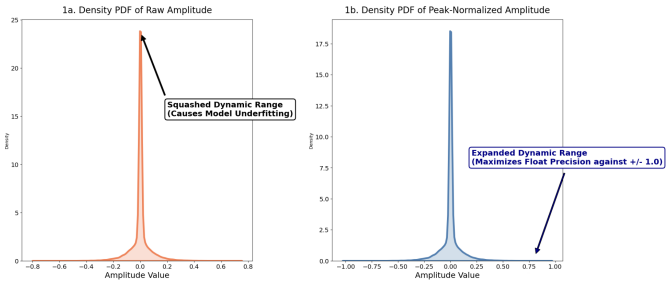

Fig. 3. (a) Raw vs. preprocessed waveform; (b) Spectrograms of clean vs. noise-augmented audio.

TABLE IV
SVM MODEL CONFIGURATION

Parameter	Value
Kernel	RBF
C search	{0.1, 1, 10, 100}
γ search	{0.001, 0.01, 0.1, 1}
Class weights	`balanced`
Cross-validation	5-fold Stratified
Scoring metric	Macro F1-Score
Scaling	`StandardScaler`

D. Phase IV: SVM Classification

The classification module (`model.py`) uses a Support Vector Machine with an RBF kernel [4].

Data Preparation: Labels are encoded using `LabelEncoder`. Data is split 80/20 (train/test) using stratified splitting. Features are scaled using `StandardScaler` (fit on training data only).

Grid Search: Best hyperparameters are found using `GridSearchCV` with 5-fold stratified cross-validation. Table IV shows the configuration.

TABLE V
SOFTWARE STACK

Library	Purpose
librosa	Audio loading, features
soundfile	Saving WAV files
scikit-learn	SVM, GridSearch, metrics
pandas	Feature CSV I/O
numpy	Numerical operations
matplotlib, seaborn	Visualization
tqdm	Progress bars

TABLE VI
OVERALL CLASSIFICATION METRICS

Metric	Value
Accuracy	23.70%
Precision (Macro)	24.54%
Recall (Macro)	29.42%
F1-Score (Macro)	26.10%
F1-Score (Weighted)	21.07%

Class Imbalance: `class_weight='balanced'` automatically penalizes minority-class misclassifications more heavily, preventing the model from always predicting the majority class.

Evaluation: Accuracy, precision, recall, and F1-score are computed. Macro averages weight all classes equally regardless of sample count.

IV. IMPLEMENTATION

The system is implemented in Python using the libraries listed in Table V.

Source Modules: `utils.py` (paths, constants), `preprocessing.py` (signal conditioning), `noise_augmentation.py` (noise injection), `feature_extraction.py` (feature computation), `model.py` (SVM training and evaluation).

Notebooks: Six sequential Jupyter notebooks (`00` through `05`) cover literature review, EDA, preprocessing, augmentation, feature engineering, and baseline ML classification.

V. RESULTS

The SVM is trained on 1,686 samples (80%) and evaluated on 422 samples (20%), with stratified splitting to preserve class proportions.

A. Overall Performance

Table VI shows the overall metrics on the test set.

B. Per-Class Results

Table VII presents the per-class precision, recall, and F1-score.

TABLE VII
PER-CLASS CLASSIFICATION RESULTS

Cry State	Prec.	Recall	F1	Support
Belly Pain	0.32	0.59	0.41	51
Burping	0.27	0.34	0.30	47
Cold/Hot	0.21	0.24	0.22	46
Discomfort	0.17	0.25	0.21	55
Hungry	0.14	0.06	0.08	153
Lonely	0.00	0.00	0.00	4
Scared	0.64	0.64	0.64	11
Tired	0.22	0.24	0.23	55
Macro Avg	**0.25**	**0.29**	**0.26**	**422**
Weighted Avg	**0.21**	**0.24**	**0.21**	**422**

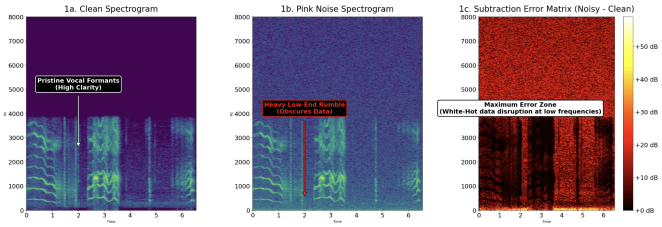

Fig. 4. Confusion matrix on the test set (422 samples).

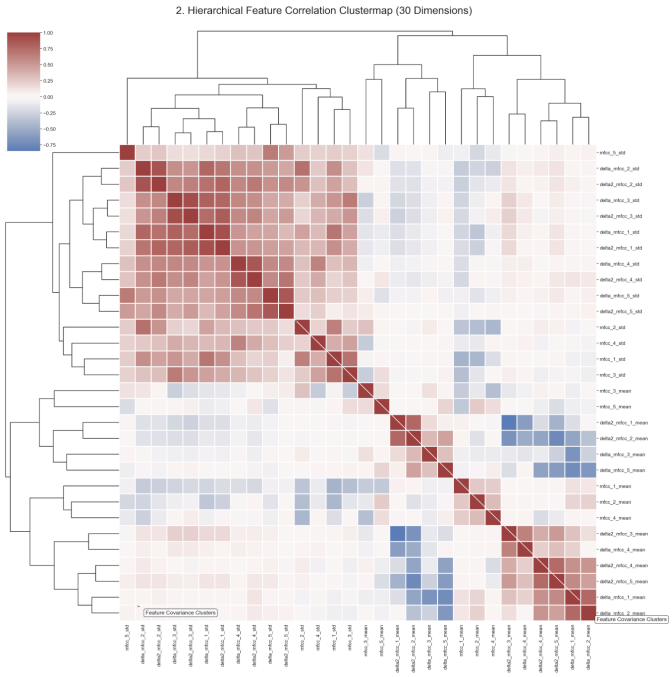

Fig. 5. ROC curves with per-class AUC scores.

C. Discussion

The baseline results show modest performance, which is expected given the challenges of this task:

1) Severe class imbalance: The *hungry* class (764 samples) dominates the dataset, yet the balanced weighting causes the

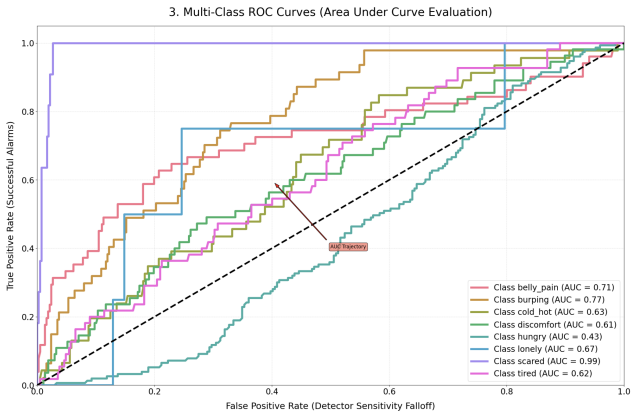

Fig. 6. PCA projection of the feature space with SVM decision boundaries.

SVM to under-predict it (recall of only 0.06), redistributing predictions toward smaller classes. *Lonely* (22 samples, 4 in test) has zero predictions entirely.

2) Class overlap: Many cry states (discomfort, cold/hot, tired) have acoustically similar characteristics, making them inherently difficult to separate with spectral features alone. The confusion matrix shows significant cross-predictions between these classes.

3) Best per-class result: *Scared* achieves the highest F1-score (0.64), suggesting it has a more distinctive acoustic signature compared to other states.

4) Interpretability advantage: Despite the modest accuracy, the transparent feature set allows us to analyze exactly which acoustic properties drive predictions, enabling targeted improvement.

Limitations:

- Small dataset size (1,054 unique recordings).
- Extreme minority classes (*lonely*: 22, *scared*: 54) have unreliable statistics.
- Handcrafted features may miss discriminative patterns that data-driven methods could capture.
- This is a Phase 1 baseline; deep learning architectures may improve results significantly.

VI. Conclusion

We built a complete pipeline for classifying infant cry states into 8 categories using classical signal processing and machine learning. The pipeline preprocesses raw audio (resampling, normalization, silence trimming), augments it with noise for robustness, extracts 106 interpretable acoustic features, and classifies them using an SVM with RBF kernel optimized via grid search.

The baseline achieves a Macro F1-Score of 0.26, with *scared* being the best-classified state (F1 = 0.64) and *lonely* being undetectable due to extreme data scarcity. The main value of this approach is interpretability—every feature has a clear physical meaning, and the model's behavior can be analyzed through confusion matrices and PCA projections. This establishes a transparent baseline for future improvements with larger datasets or hybrid feature approaches.

References

[1] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.

[2] "ECAPA-TDNN for infant cry emotion recognition," in *Proc. Interspeech*, 2023.

[3] "Infant cry detection using blueprint separable convolutional neural network with time-frequency recurrent neural network," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2023.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[5] R. E. Bellieni *et al.*, "Cry features reflect pain intensity in term newborns: An alarm threshold," *Pediatric Research*, vol. 55, no. 1, pp. 142–146, 2004.